The client will present its device identifier as the PSK identity field during the handshake, which you can then use to find the associated device profile and encrypted PSK to use to authenticate the session. Listing 13.5 shows a new `DeviceIdentityManager` class that you can use with Bouncy Castle instead of the existing PSK identity manager. The new identity manager performs a lookup in the device database to find a device that matches the PSK identity supplied by the client. If a matching device is found, then you can decrypt the associated PSK from the device profile and use that to authenticate the TLS connection. Otherwise, return `null` to abort the connection. The client doesn't need any hint to determine its own identity, so you can also return `null` from the `getHint()` method to disable the ServerKeyExchange message in the handshake just as you did in chapter 12. Create a new file named DeviceIdentityManager.java in the same folder as the Device.java file you created earlier and add the contents of the listing.

Listing 13.5 The device IdentityManager

```
package com.manning.apisecurityinaction;
import org.bouncycastle.tls.TlsPSKIdentityManager;
import org.dalesbred.Database;
import java.security.Key;
import static java.nio.charset.StandardCharsets.UTF_8;

public class DeviceIdentityManager implements
TlsPSKIdentityManager {
    private final Database database;    #A
    private final Key pskDecryptionKey;    #A
```